

P1907R0: Inconsistencies with non-type template parameters

Introduction

Non-type template parameters were originally limited to having scalar non-floating-point types. Through a number of recent changes, non-type template parameters of class type are now supported as well, provided they have strong structural equality, in particular no user-defined operator== for any of the class's subobjects.

- [P0732R2](#) Class Types in Non-Type Template Parameters
- [P1185R2](#) <=> != ==
- [P1630R1](#) Spaceship needs a tune-up, Addressing some discovered issues with P0515 and P1185

[P1714R1](#) (NTTP are incomplete without float, double, and long double!), which added floating-point types as permissible non-type template parameters (with bit-wise comparison on the value representation), was rejected by plenary straw poll in Cologne.

We should strive to remove or avoid the following categories of inconsistencies:

- (A) The run-time result of `x == y` differs from the compile-time result of `std::is_same_v<A<x>, A<y>>` for a suitably chosen template `A`.
- (B) The behavior of `x` as a top-level template argument differs from its behavior as a member of a class, a value of which is used as such a template argument.

This paper explores those inconsistencies, caused by the symbolic representation of template arguments vs. their bit-pattern runtime value, and suggests remediations.

Changes

(initial revision)

Review of inconsistencies

Here are the currently known inconsistencies, with a classification into the two categories above.

Pointer-to-member types

```
union U {  
    int x;  
    int y;  
};  
template<int U::*p> struct A;
```

`&U::x` and `&U::y` (obviously) denote different members of `U`. According to [temp.type] p1.3,

Two *template-ids* refer to the same class [...] if [...] their corresponding non-type *template-arguments* of pointer-to-member type refer to the same class member [...]

`A<&U::x>` and `A<&U::y>` are therefore different types.

However, [expr.eq] p4.5 says about run-time comparison with `==`:

If both refer to (possibly different) members of the same union (11.4), they compare equal.

Thus, `&U::x == &U::y` is true.

This is a case of both a type (A) and a type (B) inconsistency, because a pointer-to-member used as a class data member in a template argument will be compared with `==`, but as a top-level template argument, it will use the special "same class member" rule quoted above.

Floating-point types

Floating-point types cannot be used as non-type template parameters and cannot be members of classes usable as types for non-type template parameters, because built-in `<=>` comparison yields `std::partial_ordering` (see [expr.spaceship] p4.3).

With [P1714R1](#) (NTTP are incomplete without float, double, and long double!) applied, values of floating-point type would have been permissible as a top-level template argument, but not as a class member. Thus, a type (B) inconsistency would exist because the use as a class member is ill-formed. Further, a type (A) inconsistency would be introduced. For example:

```
template<float x> struct A;
static_assert(!std::is_same_v<A<+0.0>, A<-0.0>> // holds per P1714R1; different value representations
constexpr bool b = +0.0 == -0.0;                // true according to IEEE floating-point
```

References

References can appear as top-level non-type template parameters, with the following "same type" semantics:

Two *template-ids* refer to the same class [...] if [...] their corresponding non-type template-arguments of reference type refer to the same object or function [...]

According to [class.compare.default] p2, the defaulted `operator==` is defined as delete if a class contains a reference data member:

A defaulted comparison operator function for class C is defined as deleted if any non-static data member of C is of reference type or C is a union-like class (11.4.1).

Thus, such a class is unusable as a non-type template parameter for lack of strong structural equality, introducing a type (B) inconsistency.

A type (A) inconsistency is present, because `x == y` obviously does not consider whether x and y refer to the same object, but simply applies the lvalue-to-rvalue conversion, erasing any notion of object identity.

Types with user-defined `operator==`

A type with a user-defined `operator==` may exhibit a type (A) inconsistency, depending on the definition of `operator==`. Both class types and enum types may have a user-defined `operator==`. In the status quo, an absent or non-defaulted `operator==` for a class type prevents the use of that class as the type of a non-type template parameter. There is currently no comparable rule for enums, which leads to some underspecification (see [P1837R0](#) Remove NTTPs of class type from C++20 by Arthur O'Dwyer).

Suggestions

(1) Divorce template argument equality from `operator==`

If we would define template argument equality for classes as a special kind of equality, unrelated to `operator==`, we could apply the special equality rules in [temp.type] p1 also to class members of the appropriate type. Ancillary restrictions, such as restricting the permissible pointer values for non-type template arguments of pointer type, already apply uniformly to class members as well as top-level arguments; see [temp.arg.nontype] p2.

Whether a class is suitable as the type of a non-type template parameter in the first place, however, still needs to be determined. The following are possible options:

- Presence of a user-defined `operator==`: After all, if a user defines a non-defaulted `operator==` somewhere, it is a clear sign the class has special equality semantics, likely causing even more surprising type (A) inconsistencies. For the specific case of enums, which cannot have a defaulted `operator==` in their definition (because the syntax doesn't allow it), a declaration of `operator==` after the first use of that enum in a non-type template parameter is considered hostile.
- No restriction: Since `operator==` is not involved in template argument equality anymore, there is no reason to refer to it in any way. Unfathomable instantiations are prevented by the rules on permissible template arguments; see [temp.arg.nontype] p2.
- Trivial copyability: If a type is trivially copyable, its value is preserved by `memcpy`. However, trivial copyability might not be related to comparison semantics.

Regardless of the restriction chosen, the approach avoids any type (B) inconsistencies, but introduces more type (A) inconsistencies. However, type (A) inconsistencies appear to be historically acceptable, given the long-standing existing rules on pointer-to-members and references.

(2) Prohibit "bad" types as class members

We already prohibit references as members of classes for non-type template parameters, and we could do the same for pointer-to-member types. This would introduce another type (B) inconsistency, but (similar to the reference and floating-point cases) one where one of the outcomes is an ill-formed program.

Recommendation

Let's go with (1).